

CS 305: Computer Networks

Fall 2025

Lecture 10: Network Layer – The Data Plane

Ming Tang

Department of Computer Science and Engineering
Southern University of Science and Technology (SUSTech)

Chapter 4: outline

4.1 Overview of Network layer

- data plane
- control plane

4.2 What's inside a router

4.3 IP: Internet Protocol

- datagram format
- fragmentation
- IPv4 addressing
- network address translation
- IPv6

4.4 Generalized Forward and SDN

- match
- action
- OpenFlow examples of match-plus-action in action

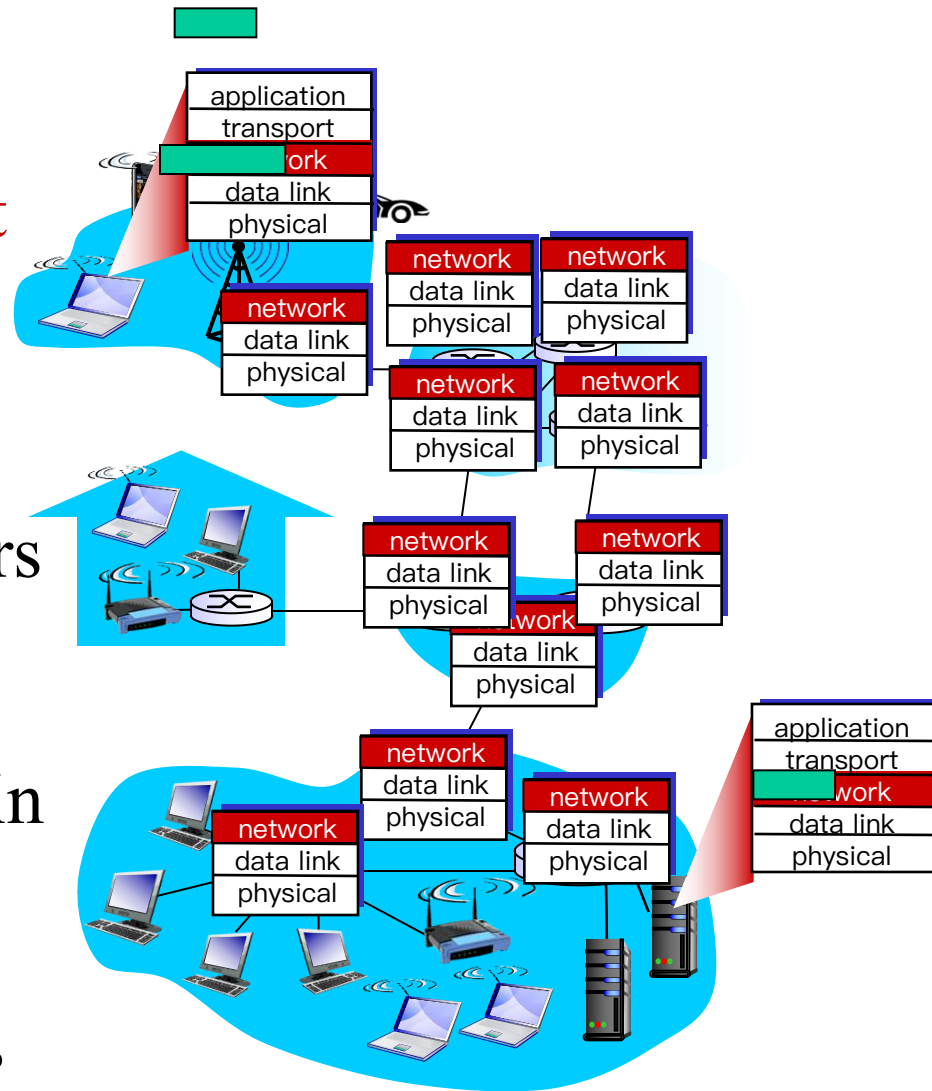
Chapter 4: network layer

Chapter goals:

- Understand principles behind network layer services, focusing on data plane:
 - network layer service models
 - forwarding versus routing
 - how a router works
 - generalized forwarding
- Instantiation, implementation in the Internet

Network layer

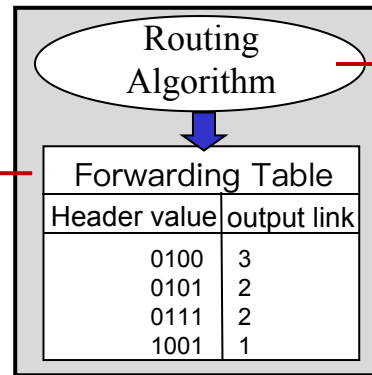
- transport **segment** from sending to receiving **host**
- on sending side **encapsulates segments into datagrams**
- on receiving side, delivers segments to transport layer
- network layer protocols in **every host, router**
- router examines header fields in all IP datagrams passing through it



Two key network-core functions

Forwarding:

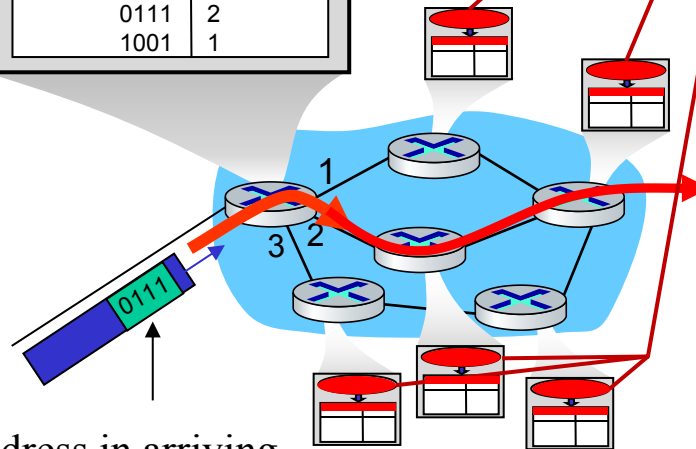
local action: move arriving packets from router's input link to appropriate router output link



destination address in arriving packet's header

Routing:

- **global** action: determine source-destination paths taken by packets
- routing algorithms



Two key network-layer functions

Network-layer functions:

- *forwarding*: move packets from router's input to appropriate router output

- **Data plane**

- *routing*: determine route taken by packets from source to destination

- routing algorithms

- **Control plane**

Analogy: taking a trip

- *forwarding*: process of getting through single interchange

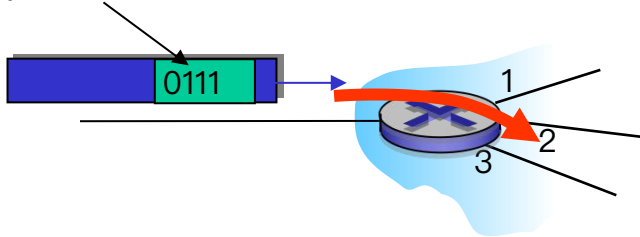
- *routing*: process of planning trip from source to destination

Network layer: data plane, control plane

Data plane

- local, per-router function, hardware
- determines how datagram arriving on router input port is forwarded to router output port
- forwarding function

values in arriving packet header



Control plane

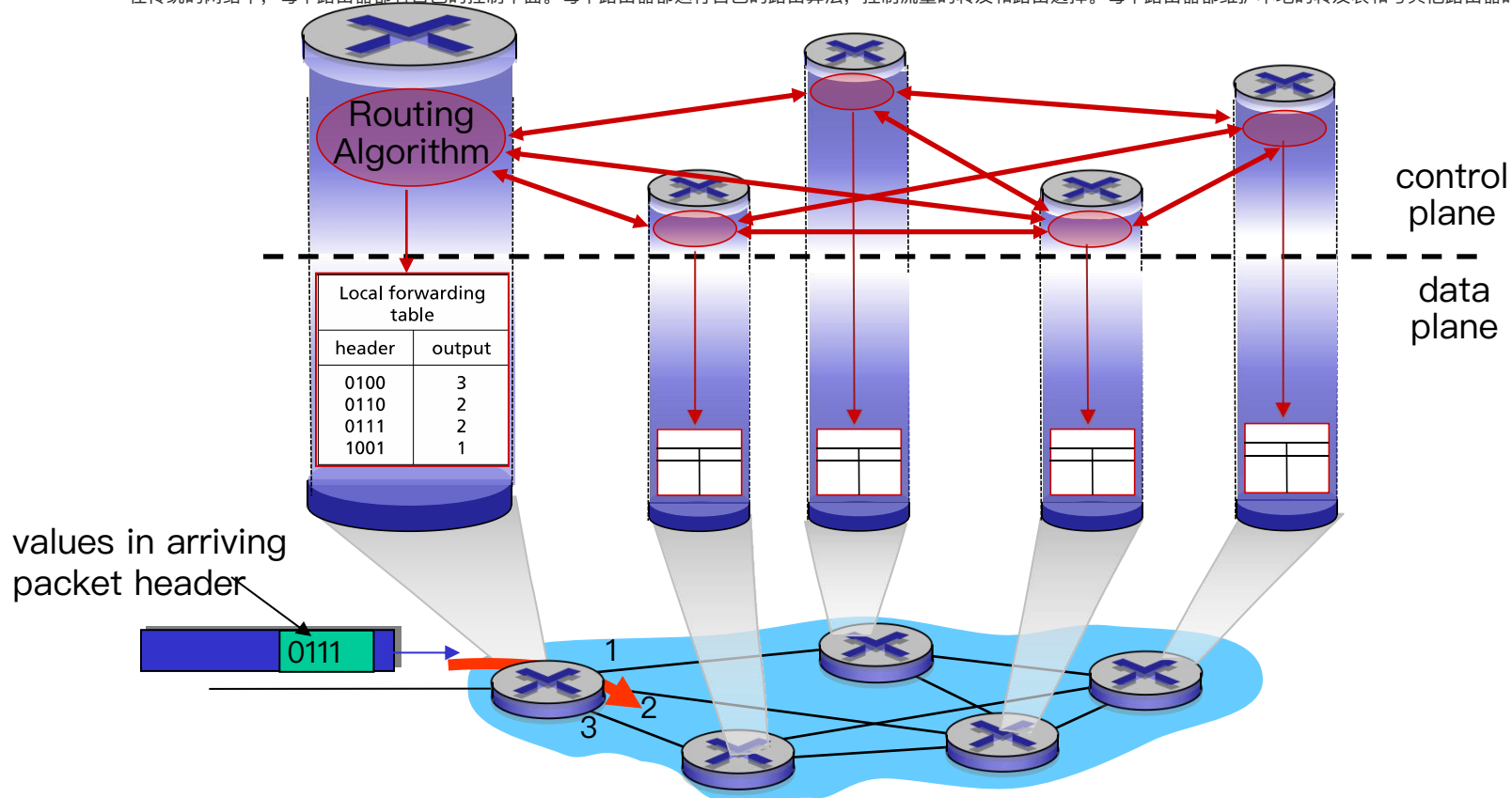
- network-wide logic, software
- determines how datagram is routed among routers along end-end path from source host to destination host
- two control-plane approaches:
 - *traditional routing algorithms*: implemented in routers
 - *software-defined networking (SDN)*: implemented in (remote) servers

Control plane: Traditional Approach

Per-router control plane: Individual routing algorithm components *in each and every router* interact in the control plane

- Forwarding and routing functions are contained within a router

在传统的网络中，每个路由器都有自己的控制平面。每个路由器都运行自己的路由算法，控制流量的转发和路由选择。每个路由器都维护本地的转发表和与其他路由器的通信

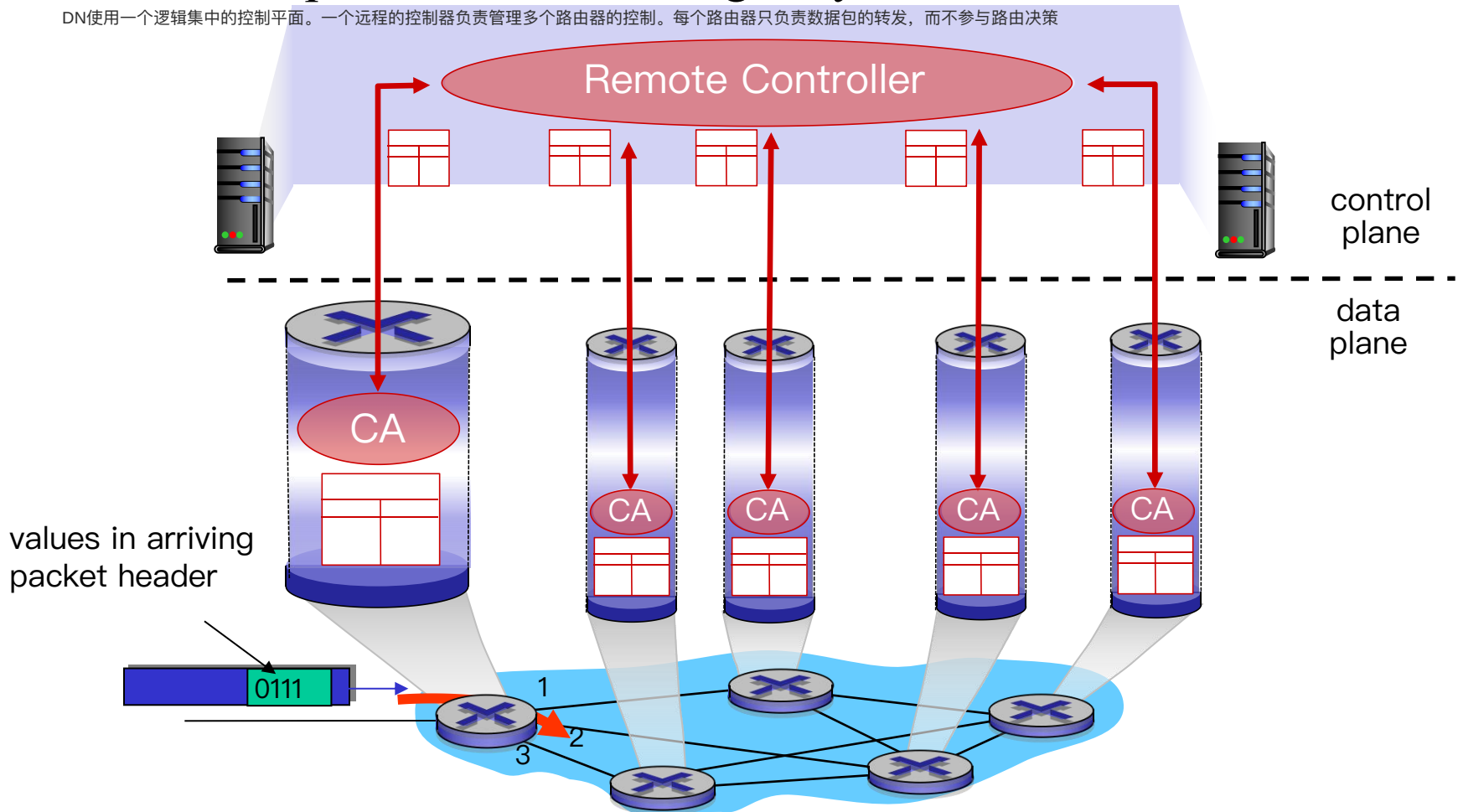


Control Plane: The SDN Approach

Logically centralized control plane: A distinct (typically remote) controller interacts with local control agents (CAs)

- Router performs forwarding only

SDN使用一个逻辑集中的控制平面。一个远程的控制器负责管理多个路由器的控制。每个路由器只负责数据包的转发，而不参与路由决策



Network service model

Q: What *service model* for “channel” transporting datagrams from sender to receiver?

example services for individual datagrams:

- guaranteed delivery
- guaranteed delivery with less than 40 msec delay

example services for a flow of datagrams:

- in-order datagram delivery
- guaranteed minimum bandwidth to flow

other example services: security

Internet service model provide “best effort” service, no guarantee on bandwidth, loss, order or timing.

Chapter 4: outline

4.1 Overview of Network layer

- data plane
- control plane

4.2 What's inside a router

4.3 IP: Internet Protocol

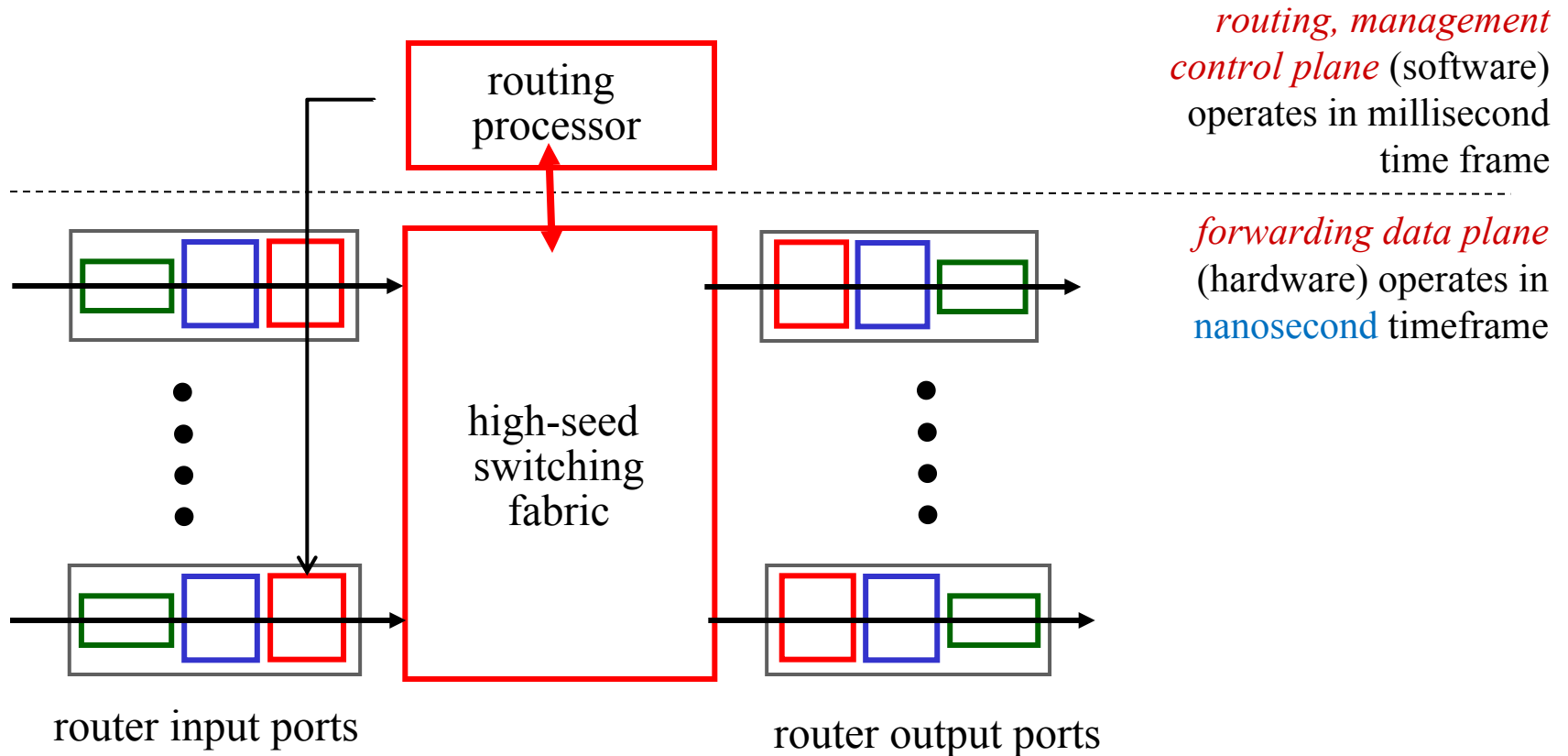
- datagram format
- fragmentation
- IPv4 addressing
- network address translation
- IPv6

4.4 Generalized Forward and SDN

- match
- action
- OpenFlow examples of match-plus-action in action

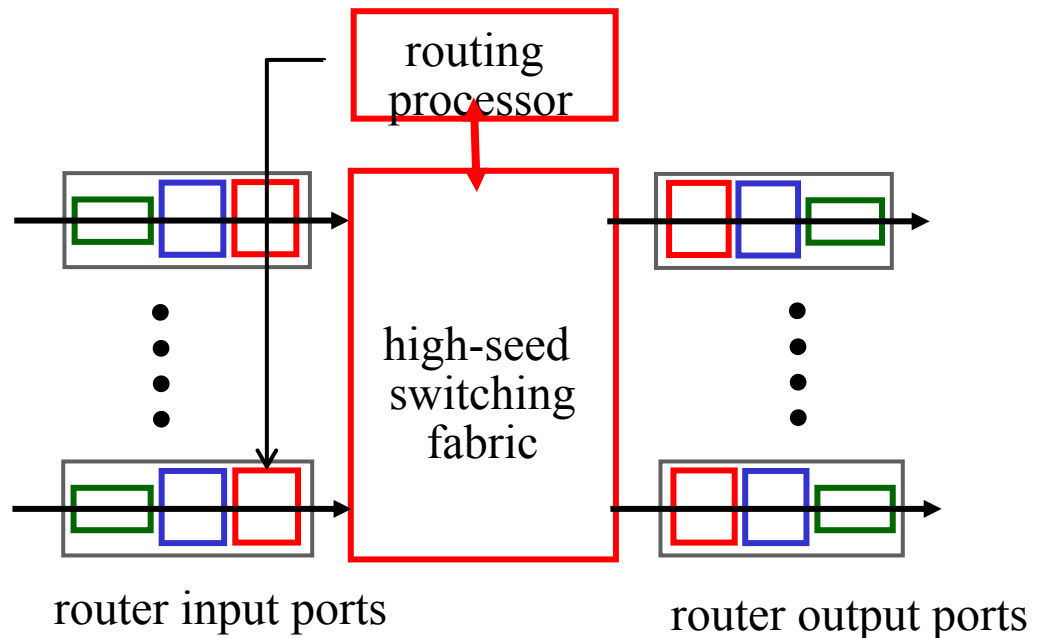
Router architecture overview

High-level view of generic router architecture:

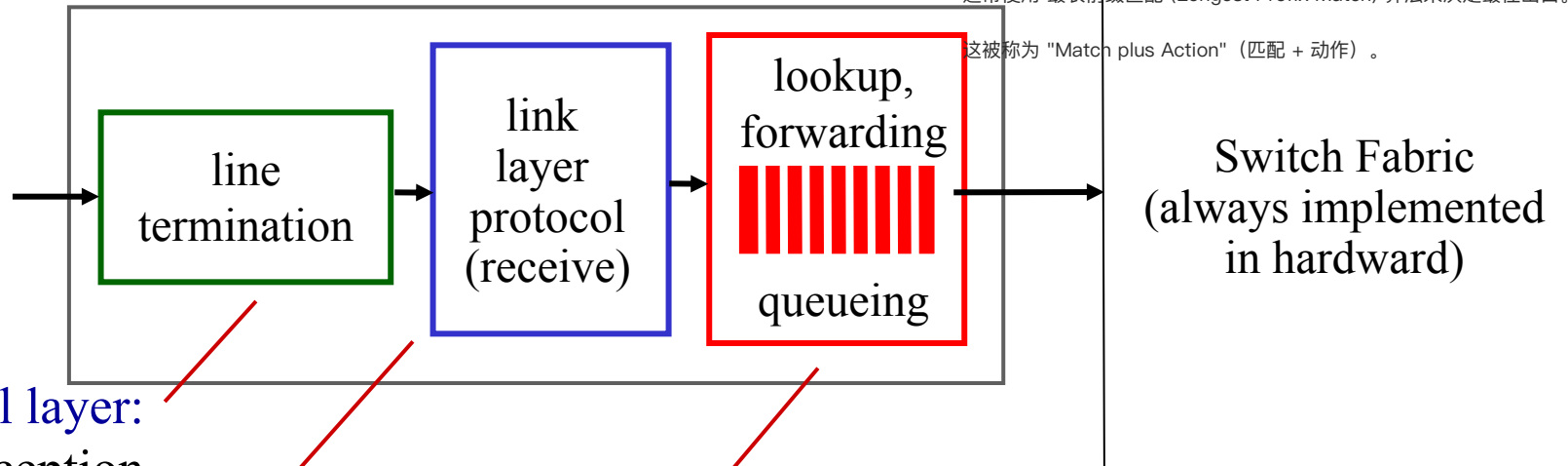


Router Overview

- **Input port**
- Switch fabrics
- Output port
- Queuing
 - Input port queue
 - Output port queue
 - Scheduling



Input port functions



查找 (Lookup):

路由器查看数据包的目的 IP 地址。

在本地内存的 转发表 (Forwarding Table) 中进行匹配。

通常使用 最长前缀匹配 (Longest Prefix Match) 算法来决定最佳出口。

这被称为 "Match plus Action" (匹配 + 动作)。

physical layer:
bit-level reception

link layer:
e.g., Ethernet
see chapter 5

decentralized switching:

- using header field values, lookup output port using forwarding table in input port memory ("*match plus action*")
- goal: complete input port processing at '**line speed**'

为什么叫分散式?

以前的老式路由器, 每个包都要交给 CPU (路由处理器) 去查表, CPU 会成为瓶颈。

现在的做法: 路由处理器计算好表后, 复制一份给每个输入端口。这样, 每个端口可以并行查表, 互不干扰。

目标: 实现 线速 (Line Speed) 转发。即: 处理数据包的速度 $\geq$ 数据包到达的

queuing: if datagrams arrive faster than forwarding rate into switch fabric

Destination-based forwarding

forwarding table

Destination Address Range	Link Interface
11001000 00010111 00010000 00000000 through 11001000 00010111 00010111 11111111	0
11001000 00010111 00011000 00000000 through 11001000 00010111 00011000 11111111	1
11001000 00010111 00011001 00000000 through 11001000 00010111 00011111 11111111	2
otherwise	3

Q: but what happens if ranges don't divide up so nicely (i.e., overlap between entities)?

Longest prefix matching

longest prefix matching

when looking for forwarding table entry for given destination address, use *longest* address prefix that matches destination address.

如果一个 IP 地址同时符合两个规则的范围，该听谁的？

这就是 最长前缀匹配 (Longest Prefix Matching, LPM) 原则的由来。

Destination Address Range	Link interface
11001000 00010111 00010*** *****	0
11001000 00010111 00011000 *****	1
11001000 00010111 00011*** *****	2
otherwise	3

examples:

DA: 11001000 00010111 00010110 10100001 0

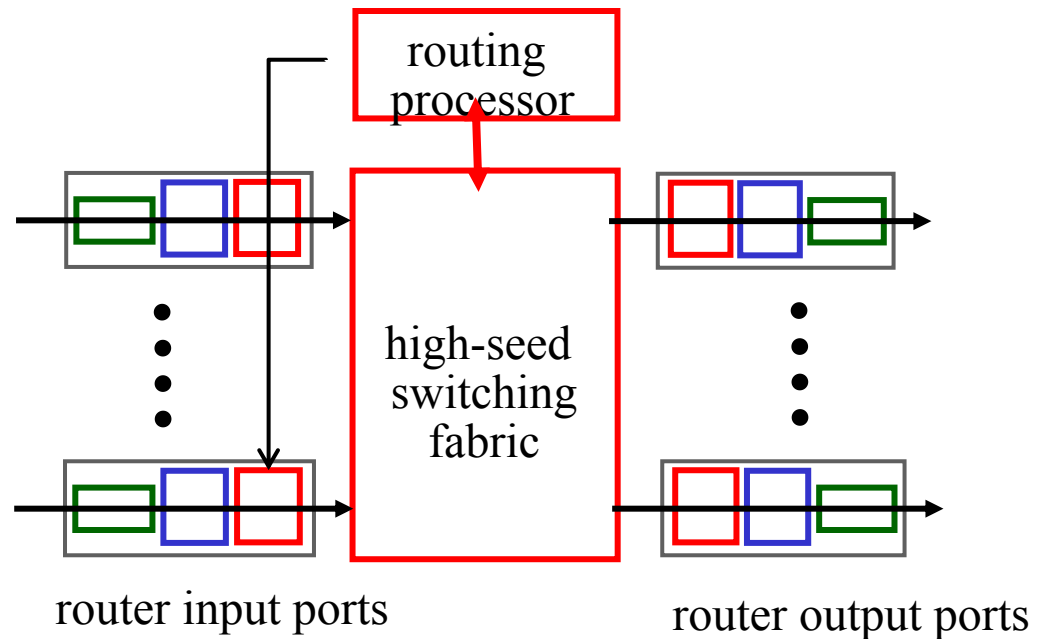
which interface?

DA: 11001000 00010111 00011000 10101010 1

which interface?

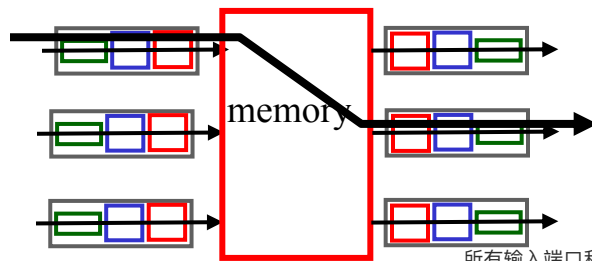
Router Overview

- Input port
- **Switching fabrics**
- Output port
- Queuing
 - Input port queue
 - Output port queue
 - Scheduling



Switching fabrics

- transfer packet from input buffer to appropriate output buffer
- switching rate: rate at which packets can be transferred from inputs to outputs
 - often measured as multiple of input/output line rate
 - N inputs: switching rate N times line rate desirable
- three types of switching fabrics

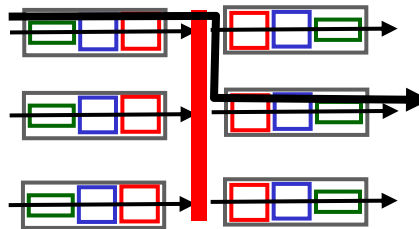


所有输入端口和输出端口都挂在一条共享的总线 (Bus) 上。

Switch via memory

- Interrupt; write and read
- Two packets cannot be forwarded at the same time

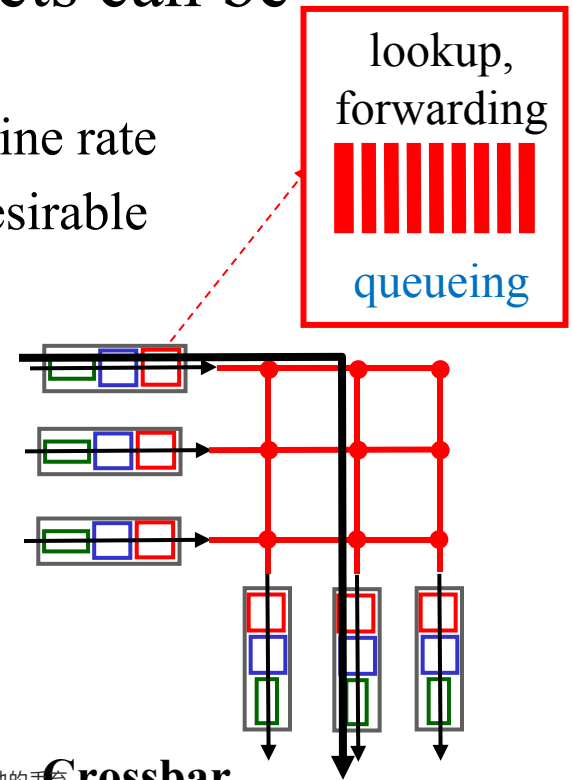
因为 CPU 和内存一次只能处理一件事。



输入端口给数据包贴个“标签” (通过内部头部), 然后扔到总线上。

Switch via bus

- Broadcast, label
- One packet can cross at a time

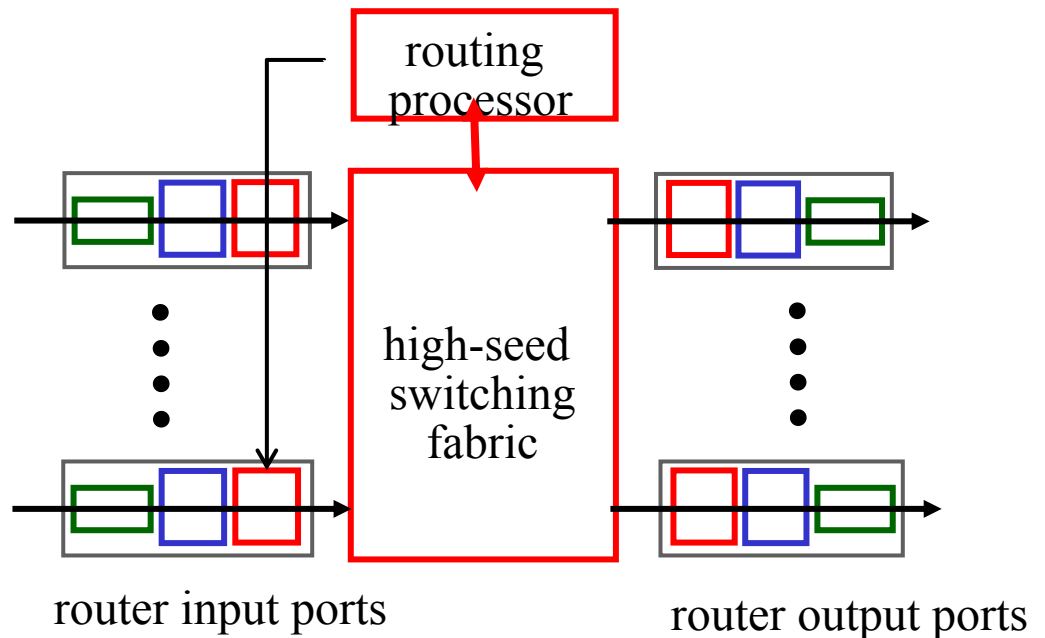


Crossbar

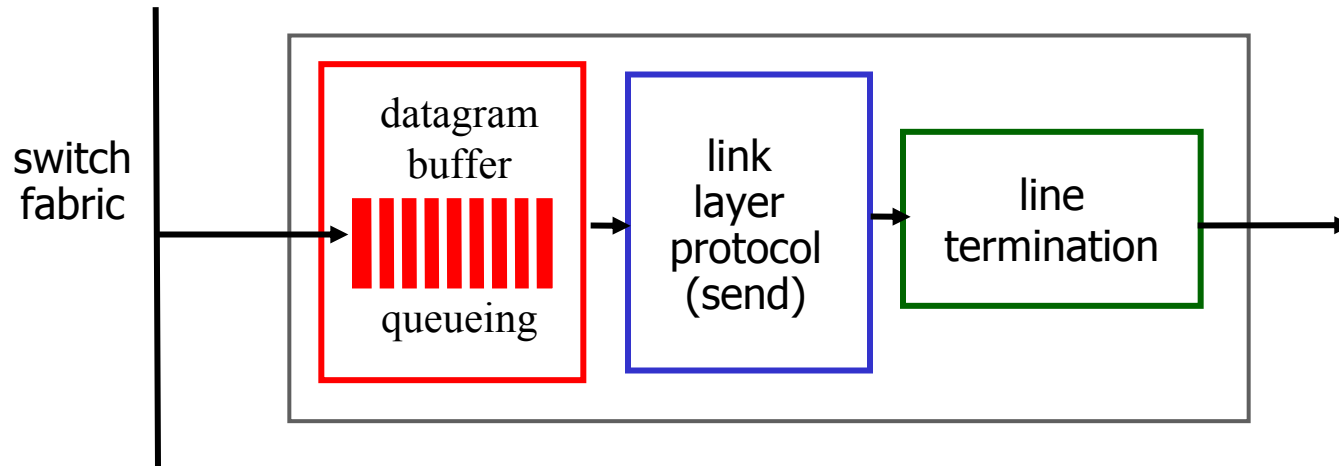
- Multiple packets in parallel
- Non-blocking

Router Overview

- Input port
- Switch fabrics
- **Output port**
- Queuing
 - Input port queue
 - Output port queue
 - Scheduling



Output ports



- *buffering* required when datagrams arrive from fabric faster than the transmission rate

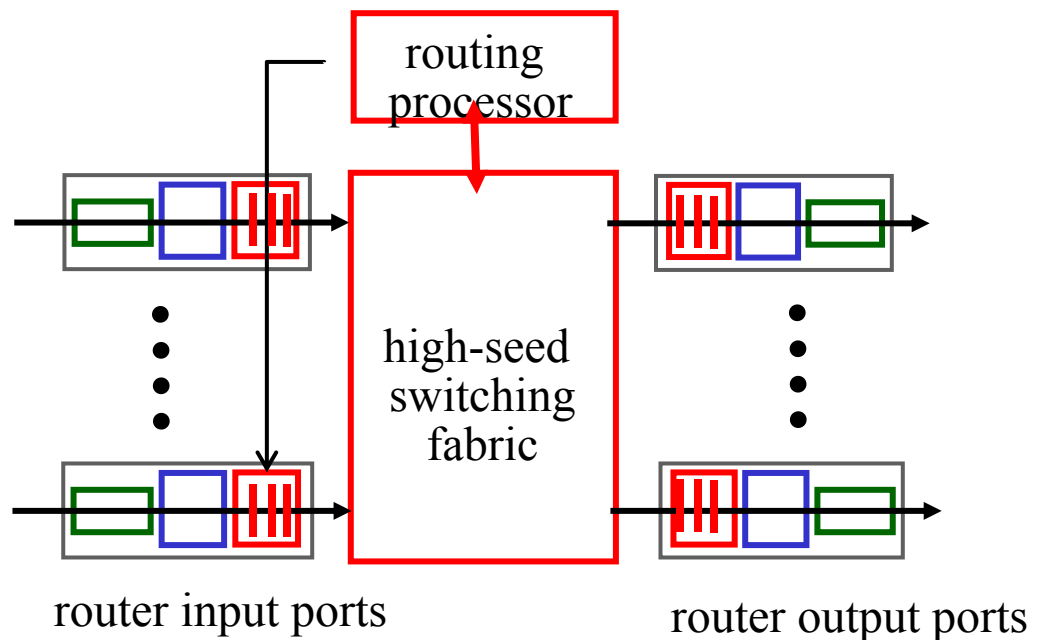
Datagram (packets) can be lost due to congestion, lack of buffers

- *scheduling discipline* chooses among queued datagrams for transmission

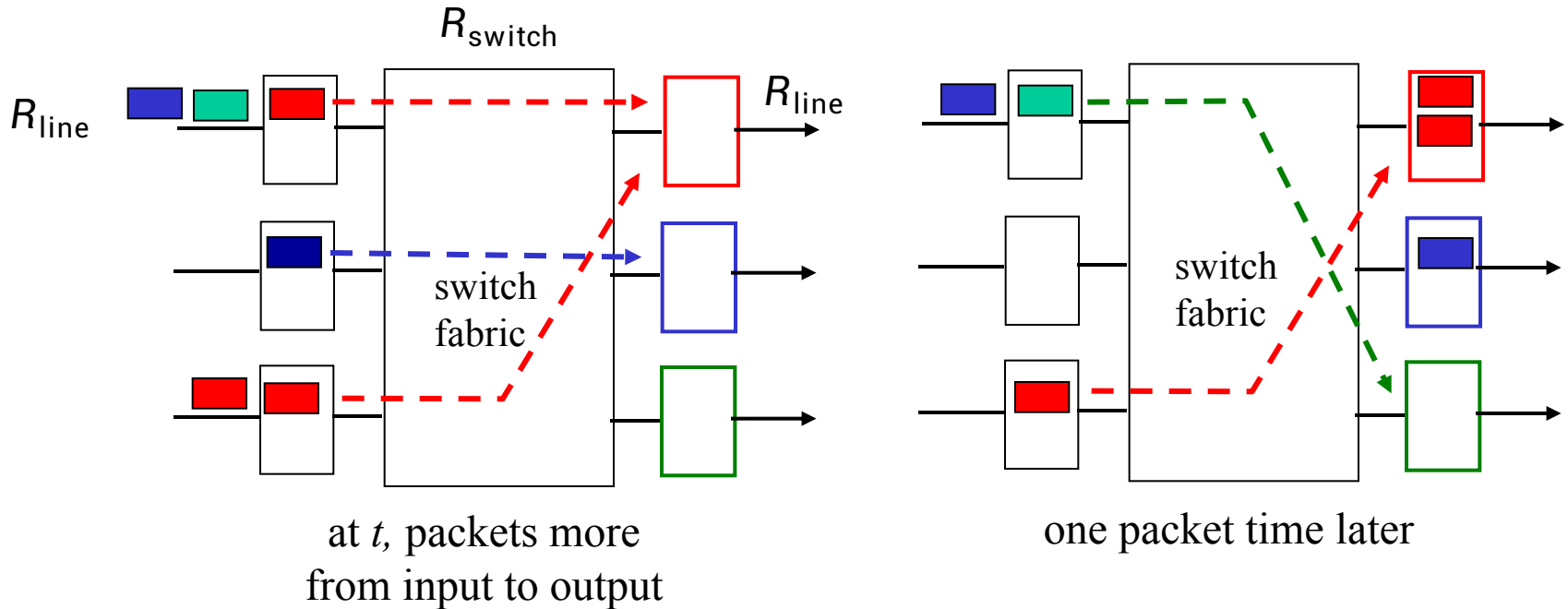
Priority scheduling – who gets best performance, network neutrality

Router Overview

- ❖ Input port
- ❖ Switch fabrics
- ❖ Output port
- ❖ **Queuing**
 - Input port queue
 - Output port queue
 - Queue scheduling



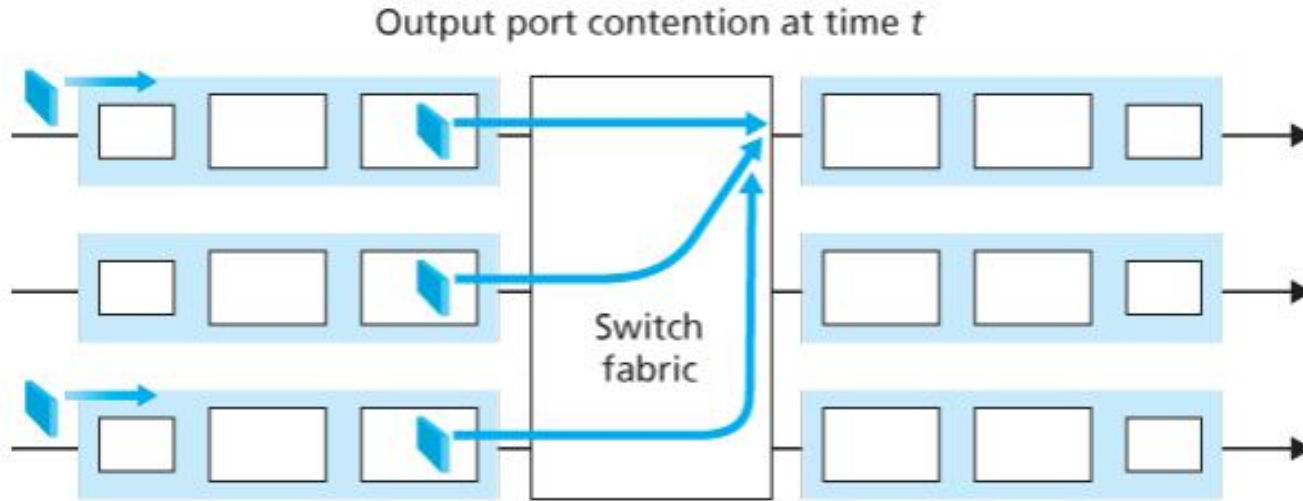
Input port queueing



Switch fabric is not fast enough (e.g., suppose $R_{switch} = R_{line}$)

- Packet queuing occur at **input port**
- Crossbar, and **multiple packets** must be transferred to the **same port**

Output port queueing



- If R_{switch} is N times faster than R_{line} , then negligible queuing at the input ports.
- buffering when arrival rate via switch exceeds output line speed
- *queueing (delay) and loss due to output port buffer overflow!*

How much buffering?

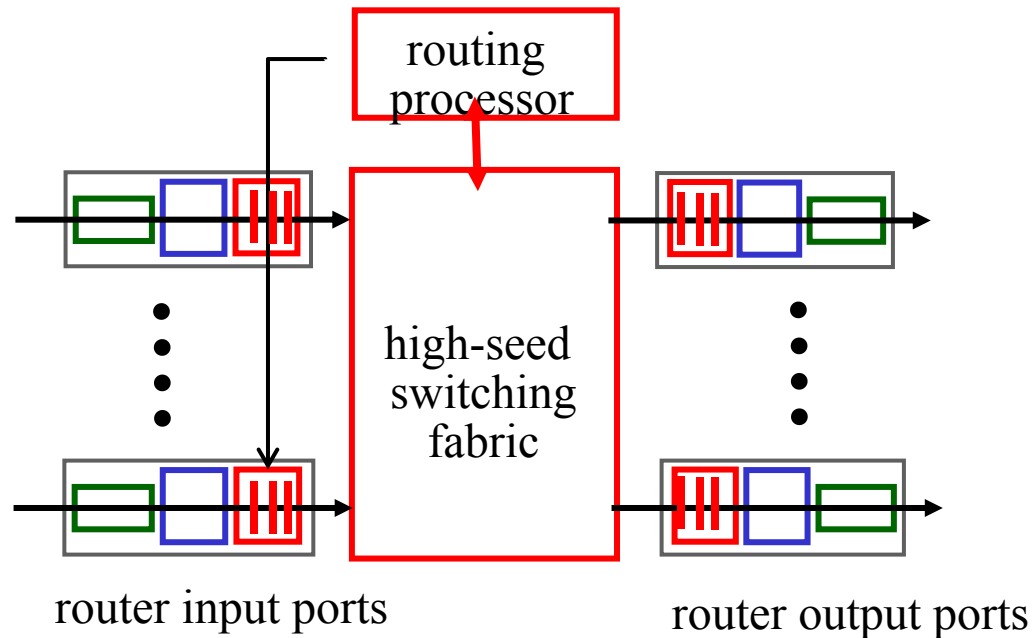
- RFC 3439 rule of thumb: average buffering equal to “typical” RTT (say 250 msec) times link capacity C
 - e.g., $C = 10$ Gpbs link: 2.5 Gbit buffer
- recent recommendation: with N flows, buffering equal to

$$\frac{RTT \cdot C}{\sqrt{N}}$$

当网络中存在大量并发的流 (N flows) 时, 由于流量的平滑效应 (各种流的发送和延迟错开), 所需的缓存量可以大大减少

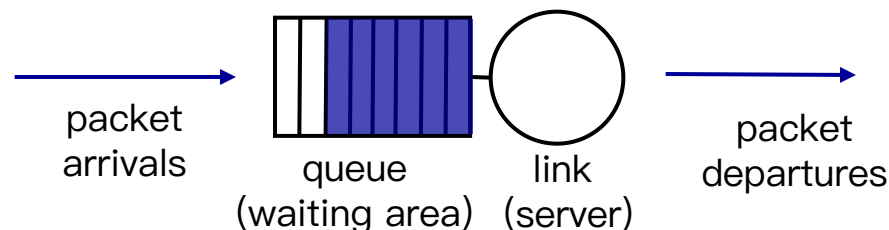
Scheduling mechanisms

- *scheduling*: choose next packet to send on link



Scheduling policies: FIFO

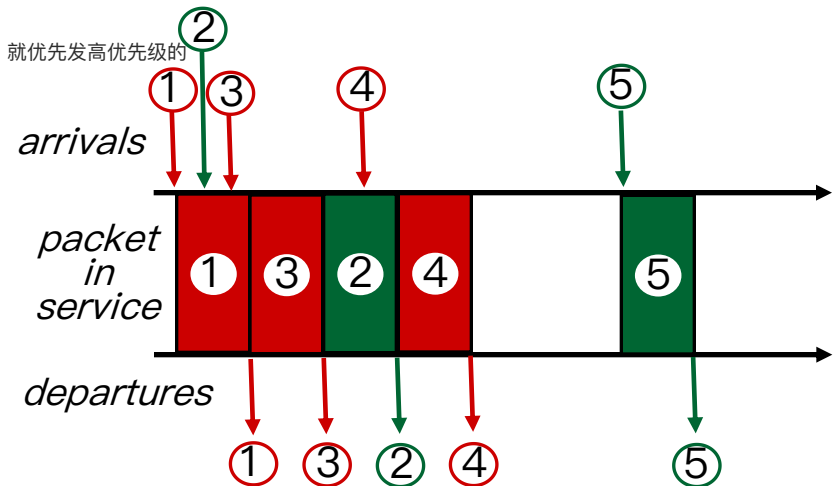
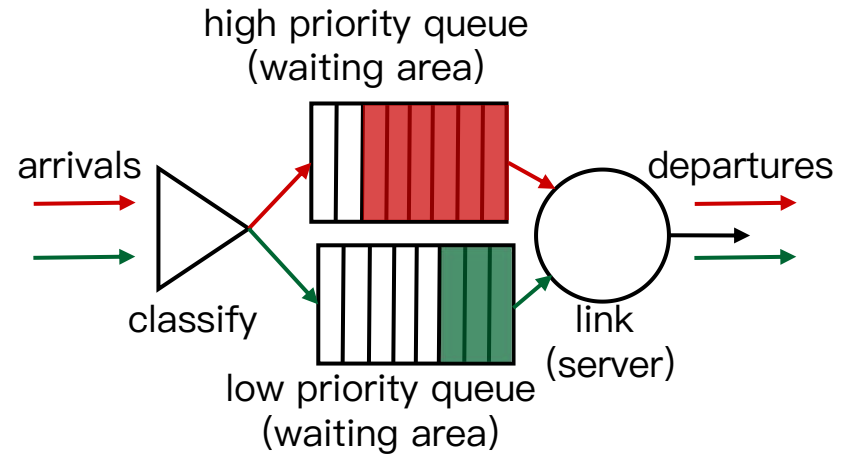
- *FIFO (first in first out) queuing*: send in order of arrival to queue
- *discard policy*: if packet arrives to full queue: who to discard?
 - *tail drop*: drop arriving packet
 - *priority*: drop/remove on priority basis
 - *random*: drop/remove randomly



Scheduling policies: priority

priority queuing: send highest priority queued packet

- multiple *classes*, with different priorities
 - class may depend on marking or other header info, e.g. IP source/dest, port numbers, etc.
- Transmit a packet from the highest priority class
- **Non-preemptive** priority queuing

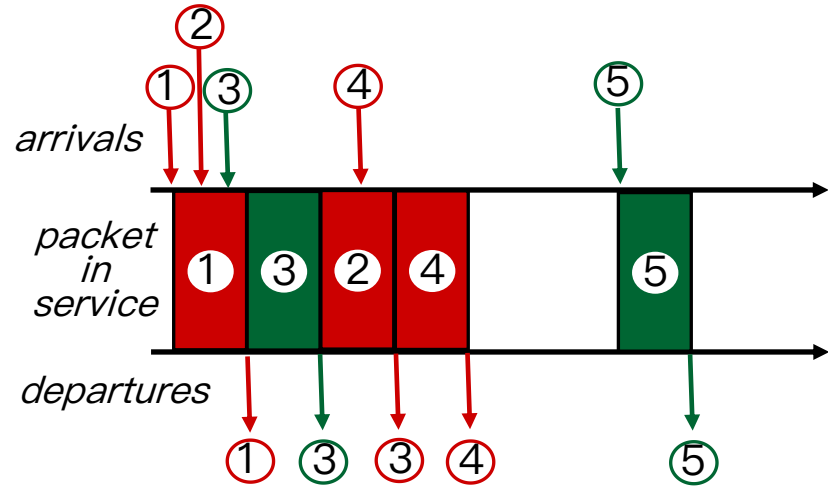
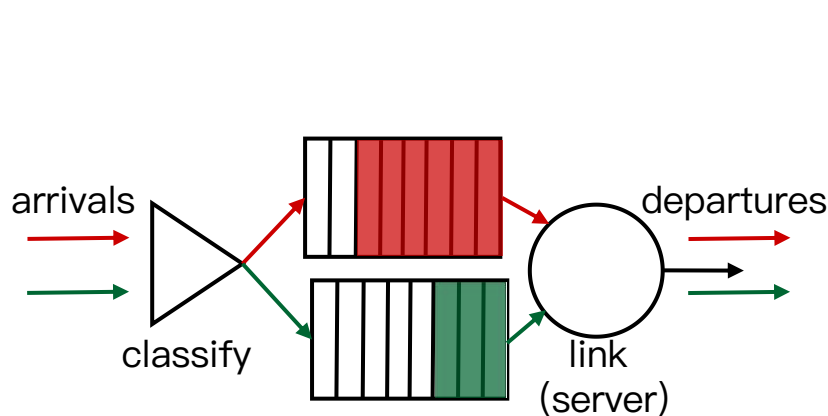


Scheduling policies: still more

Round Robin (RR) scheduling:

- multiple classes
- cyclically scan class queues, sending one complete packet from each class (if available)

If available (如果有的话) : 如果轮到红色了, 但红色队列是空的, 那就直接跳过, 去发绿色的 (不会傻等) 。

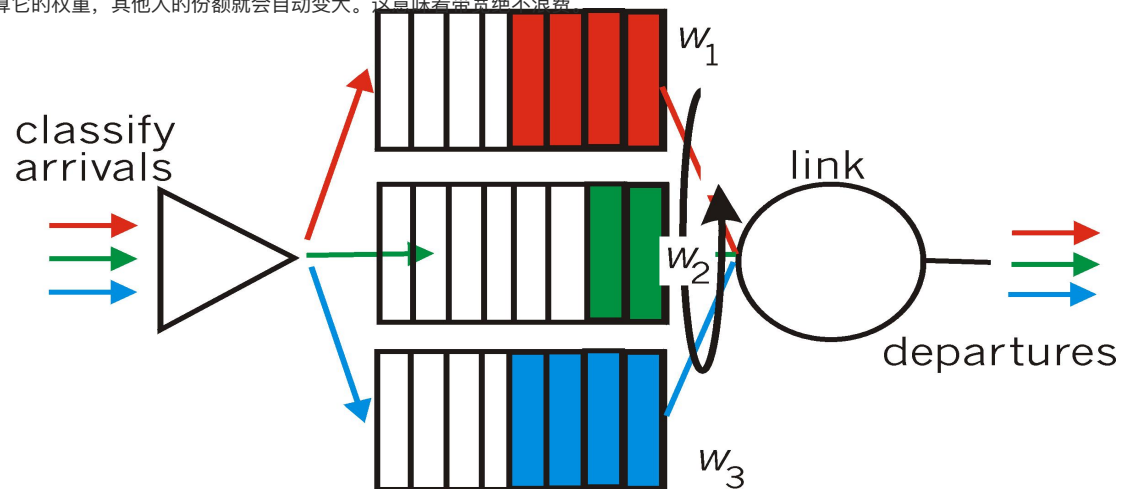


Scheduling policies

Weighted Fair Queuing (WFQ):

- generalized Round Robin
- each class gets weighted amount of service in each cycle
 - $w_i / \sum_{j \in Q^{busy}} w_j$ of the bandwidth (throughput)
 - Q^{busy} : all classes that have queued packets
 - Worst case: all queues have packets; $w_i / \sum_{j \in Q} w_j$

如果某个 VIP 队列是空的（没数据要发），那就不算它的权重，其他人的份额就会自动变大。这意味着带宽绝不浪费



Chapter 4: outline

4.1 Overview of Network layer

- data plane
- control plane

4.2 What's inside a router

4.3 IP: Internet Protocol

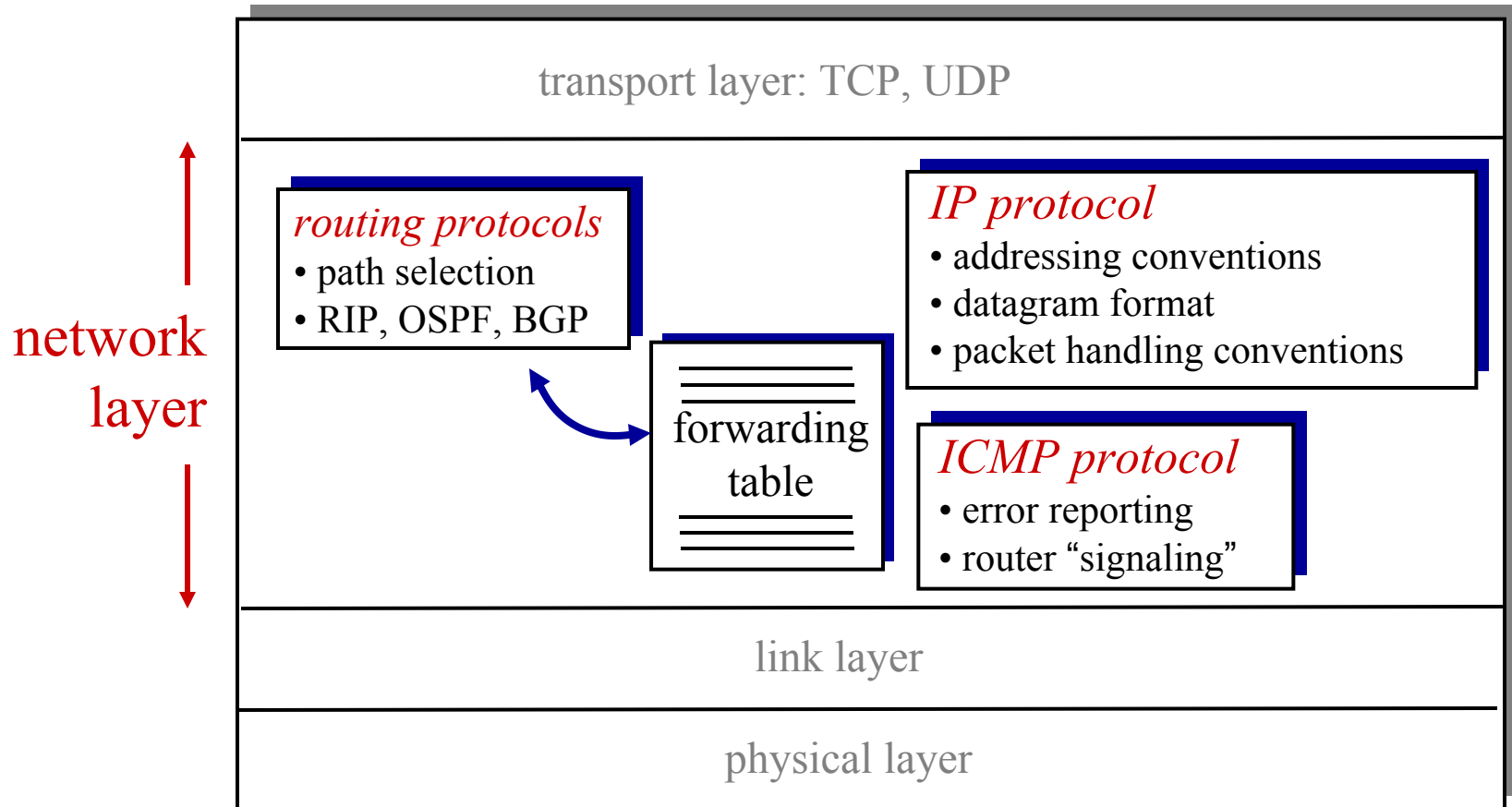
- datagram format
- fragmentation
- IPv4 addressing
- network address translation
- IPv6

4.4 Generalized Forward and SDN

- match
- action
- OpenFlow examples of match-plus-action in action

The Internet network layer

host, router network layer functions:



IP datagram format

IP protocol version
number

header length
(bytes)

“type” of data

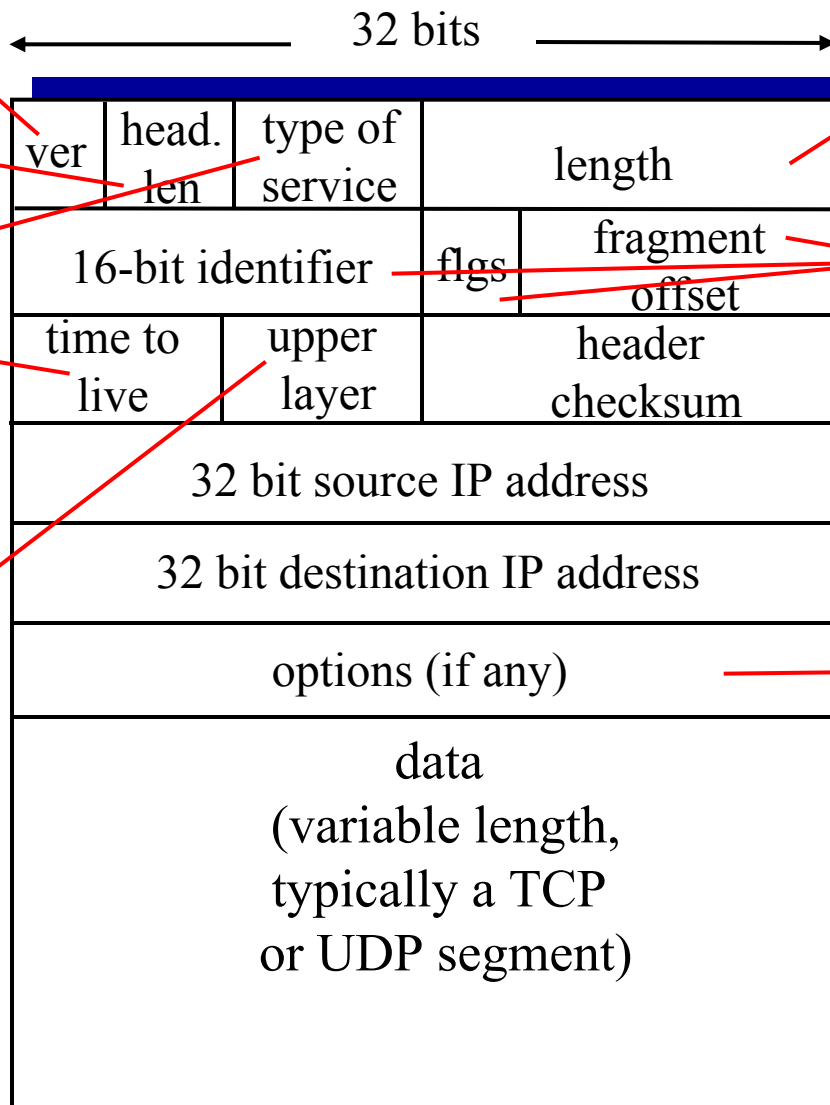
max number
remaining hops
(decremented at
each router)

upper layer protocol
to deliver payload to

\$6\$ 代表 TCP, \$17\$ 代表 UDP

how much overhead?

- ❖ 20 bytes of TCP
- ❖ 20 bytes of IP
- ❖ = 40 bytes + app layer overhead



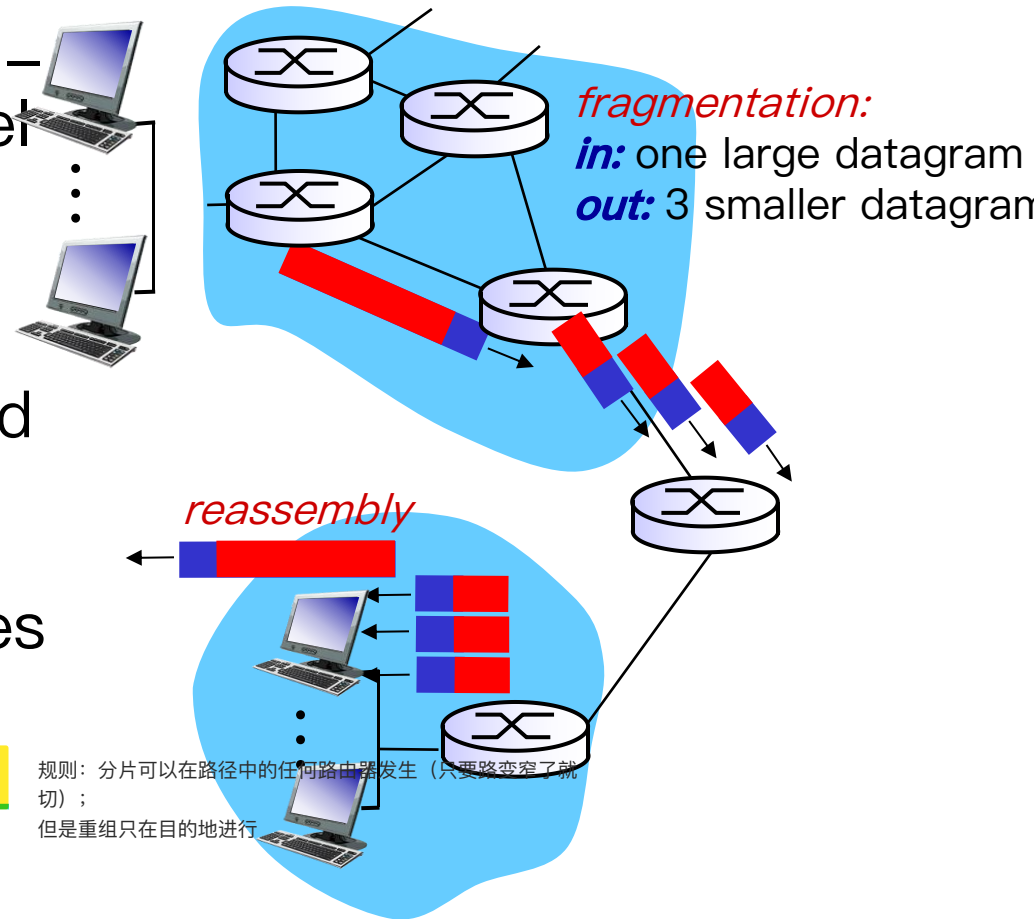
total datagram
length (bytes)

for
fragmentation/
reassembly

e.g. timestamp,
record route
taken, specify
list of routers
to visit.

IP fragmentation, reassembly

- network links have **max transmission unit (MTU)** – largest possible link-level frame
 - different link types, different MTUs
- large IP datagram divided (“fragmented”) within net
 - one datagram becomes several datagrams
 - “resembled” **only at final destination**
 - IP header bits used to identify, order related fragments



如何分片？ 利用上一张图里的那三个字段：

Identifier (标识符)：给切碎的每一片都打上同一个 ID（比如 888），这样接收方就知道它们属于同一个大包。

Flags (标志位)：用来标记“后面还有没有分片”。

最后一片的 Flag 是 0，前面的都是 1（意思是“我后面还有兄弟”）。

Offset (片偏移)：告诉接收方，这一小片应该拼在原始大包的哪个位置。

IP fragmentation, reassembly

数据包总长度 = IP头部 (20字节) + 数据载荷 (Payload)

example:

- ❖ 4000 byte datagram
- ❖ MTU = 1500 bytes

1480 bytes in data field

IP 协议规定, 偏移量的单位是 8 字节 (为了节省头部空间, 把大数字除以8存储)
前面已经发了1480bytes, 所以偏移量是1480/8

offset =
1480/8

	length	ID	fragflag	offset	
	=4000	=x	=0	=0	

one large datagram becomes several smaller datagrams

fragflag=1代表着后面还有分片

	length	ID	fragflag	offset	
	=1500	=x	=1	=0	

	length	ID	fragflag	offset	
	=1500	=x	=1	=185	

	length	ID	fragflag	offset	
	=1040	=x	=0	=370	

Chapter 4: outline

4.1 Overview of Network layer

- data plane
- control plane

4.2 What's inside a router

4.3 IP: Internet Protocol

- datagram format
- fragmentation
- IPv4 addressing
- network address translation
- IPv6

4.4 Generalized Forward and SDN

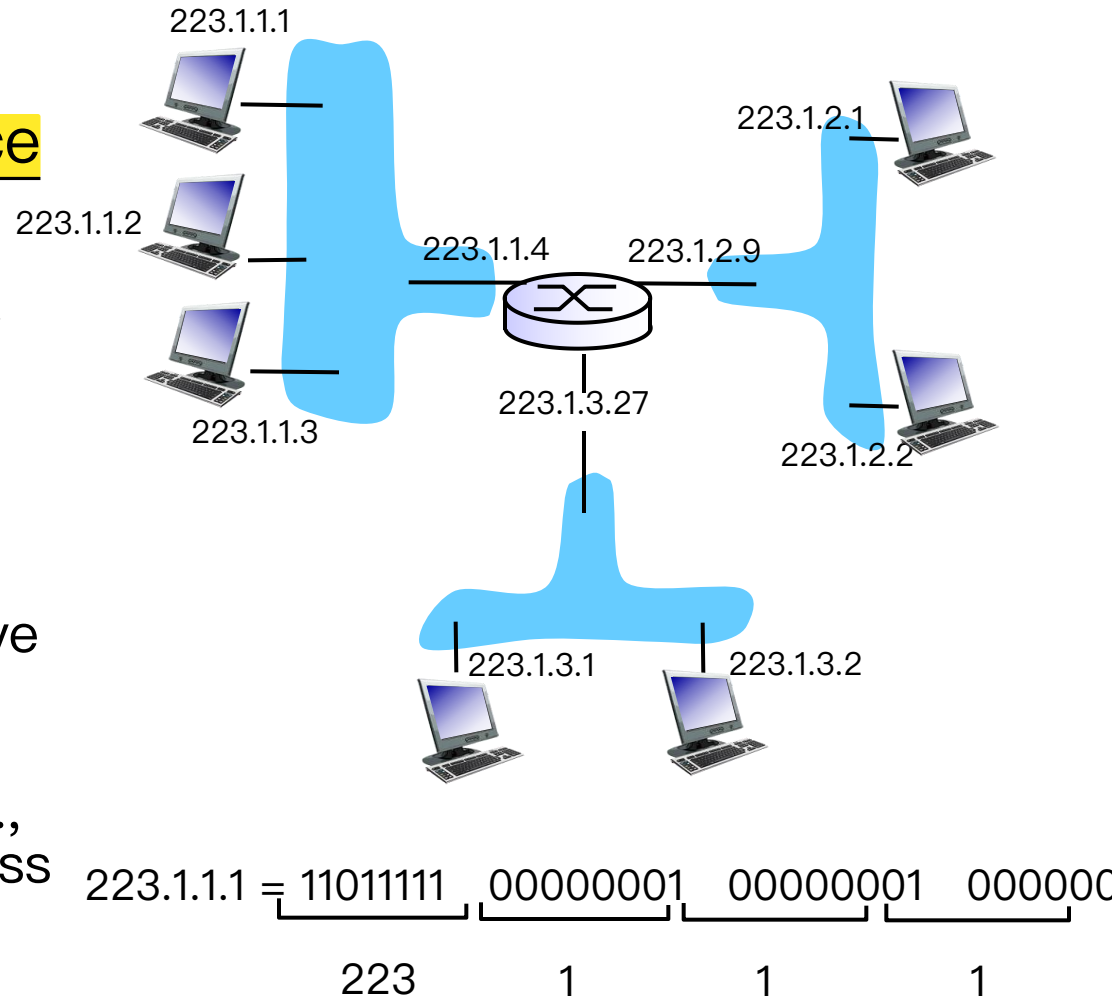
- match
- action
- OpenFlow examples of match-plus-action in action

Overview

- IP addressing
- Subnet
- How to assign/obtain IP address?

IP addressing: introduction

- *IP address*: 32-bit identifier for **interface** of hosts and routers
- *interface*: (network interface card) connection between host/router and physical link
 - router's typically have multiple interfaces
 - host typically has one or two interfaces (e.g., wired Ethernet, wireless 802.11)
- *IP addresses associated with each interface*



IP addressing: introduction

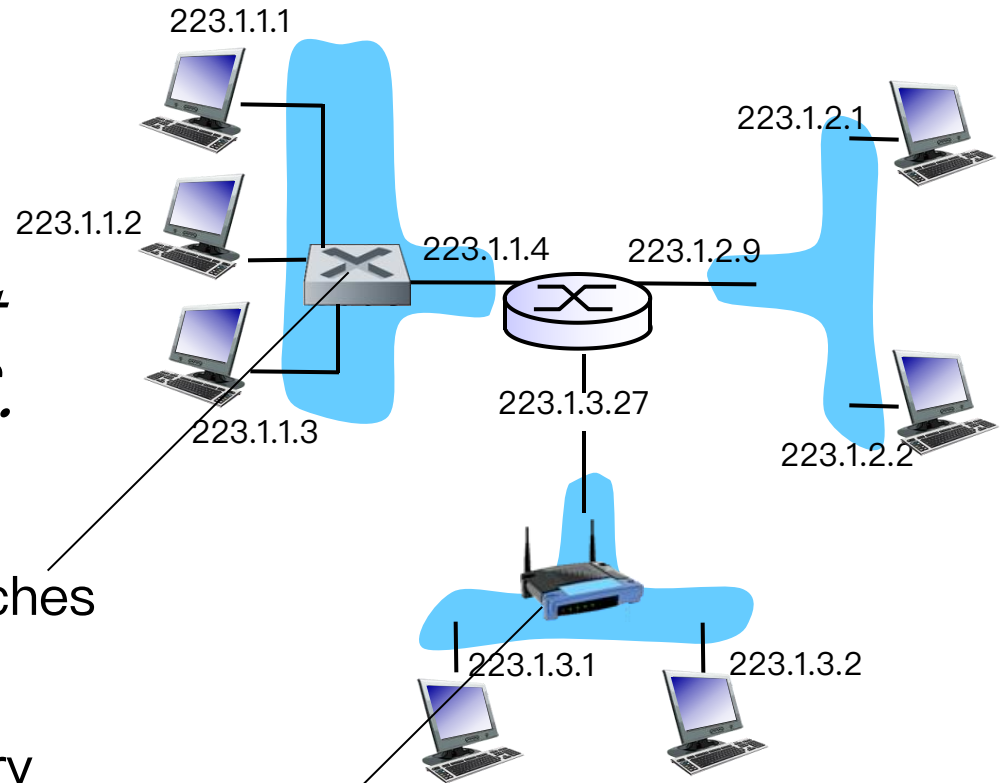
Q: how are interfaces actually connected?

A: we'll learn about that in chapter 5, 6.

A: wired Ethernet interfaces connected by Ethernet switches

For now: don't need to worry about how one interface is connected to another (with no intervening router)

A: wireless WiFi interfaces connected by WiFi base station

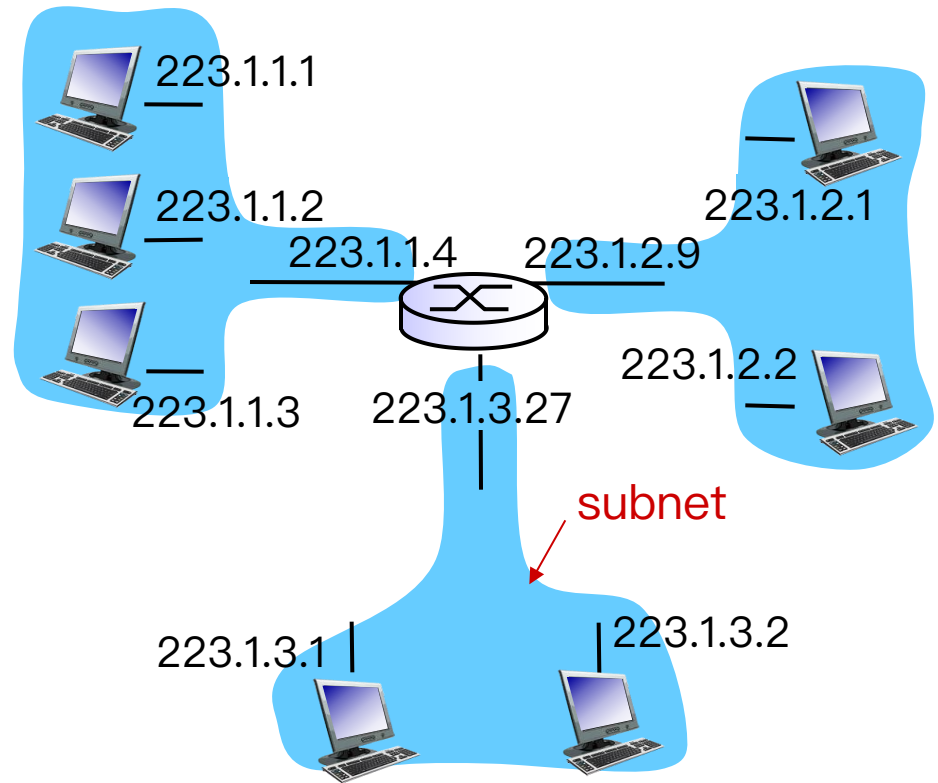


Overview

- IP addressing
- Subnet
- How to assign/obtain IP address?

Subnets

- IP address:
 - subnet part – high order bits
 - host part – low order bits
- *what's a subnet?*
 - device interfaces with same subnet part of IP address
 - can physically reach each other **without intervening router**

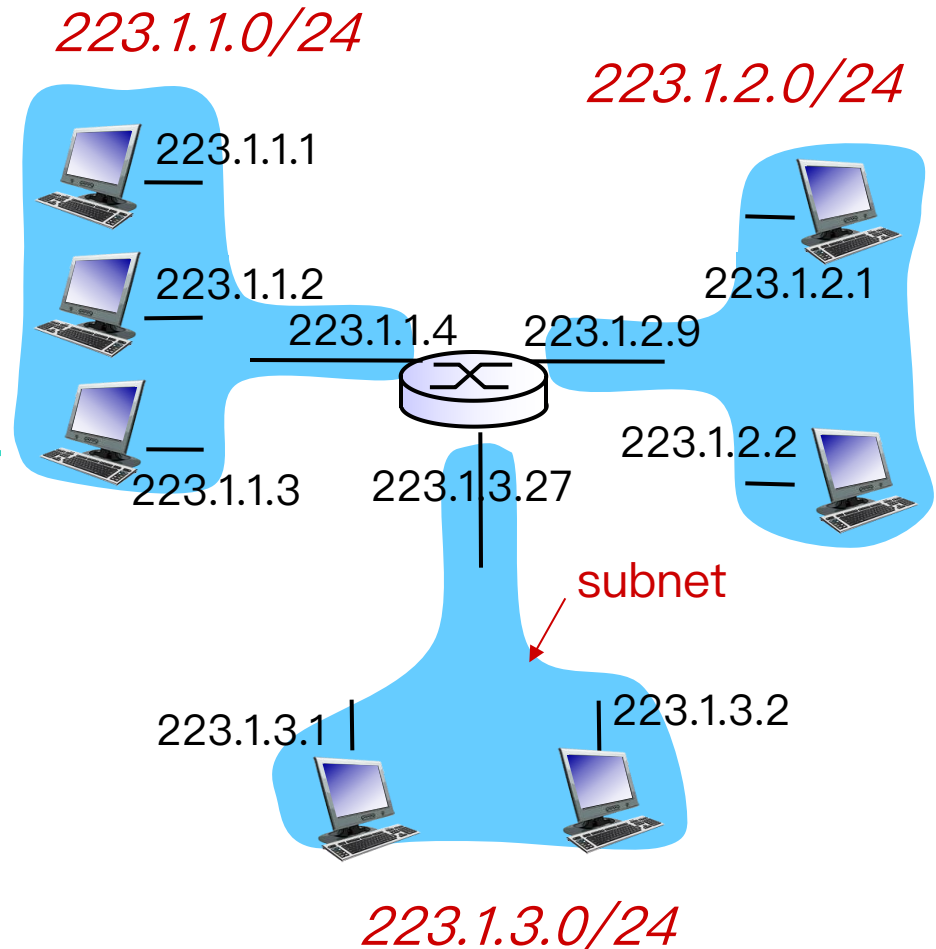


network consisting of 3 subnets

Subnets

recipe

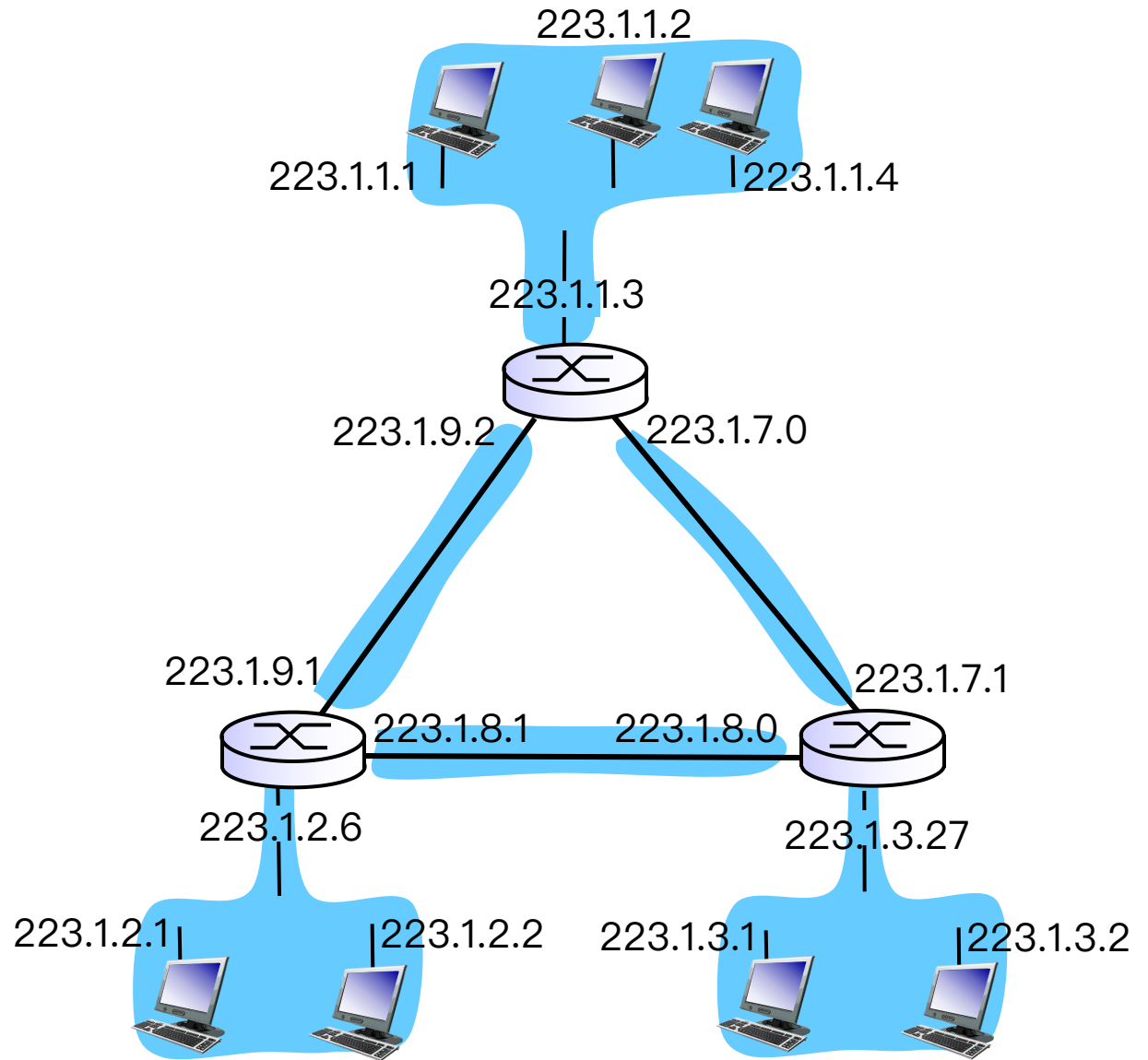
- to determine the subnets, detach each interface from its host or router, creating islands of isolated networks
- each isolated network is called a *subnet*



subnet mask: /24
255.255.255.0

Subnets

how many?



Overview

- IP addressing
- Subnet
- How to assign/obtain IP address?

IP addressing: CIDR

CIDR: Classless InterDomain Routing

- A method to assign blocks of IP address
- subnet portion of address of arbitrary length

这表示 IP 地址的前 x 位是网络号 (Network Prefix), 剩下的 32-x 位是主机号 (Host Part)。

- address format: **a.b.c.d/x**, where x is # bits in subnet portion of address

← subnet part → ← host part →
11001000 00010111 00010000 00000000

200.23.16.0/23

Subnet mask: 255.255.254.0

How does an ISP get block of addresses?

Q: how does an ISP get block of addresses?

A: ICANN: Internet Corporation for Assigned Names and Numbers

<http://www.icann.org/>

- allocates addresses
- manages DNS
- assigns domain names, resolves disputes

How does a subnet get block of address

Q: how does *network* get subnet part of IP addr?

A: gets allocated portion of its provider ISP's address space

ISP's block	<u>11001000 00010111 00010000</u> 00000000	200.23.16.0/20
Organization 0	<u>11001000 00010111 00010000</u> 00000000	200.23.16.0/23
Organization 1	<u>11001000 00010111 00010010</u> 00000000	200.23.18.0/23
Organization 2	<u>11001000 00010111 00010100</u> 00000000	200.23.20.0/23
...		
Organization 7	<u>11001000 00010111 00011110</u> 00000000	200.23.30.0/23

How does a subnet get block of address

Suppose all of the interfaces in each of these three subnets are required to have the prefix 166.4.20.128/25.

- Subnet 1 is required to support at least 62 interfaces
- Subnet 2 is required to support at least 30 interfaces
- Subnet 3 is required to support at least 28 interfaces

Provide three network addresses (of the form a.b.c.d/x) that satisfy these constraints.

How does a subnet get block of addresses

先分需求最大的，再分小的

Block of addresses: 166.4.20.128/25. Subnet 1: at least 62 interfaces; Subnet 2: at least 30 interfaces; Subnet 3: at least 28 interfaces

This block of IP addresses can be written as

10100110 00000100 00010100 10000000

Subnet 1: since $2^6 = 64 > 62$, we can assign the following block

10100110 00000100 00010100 10000000

which can be represented as 166.4.20.128/26.

Subnet 2: since $2^5 = 32 > 30$, we can assign the following block

10100110 00000100 00010100 11000000

which can be represented as 166.4.20.192/27.

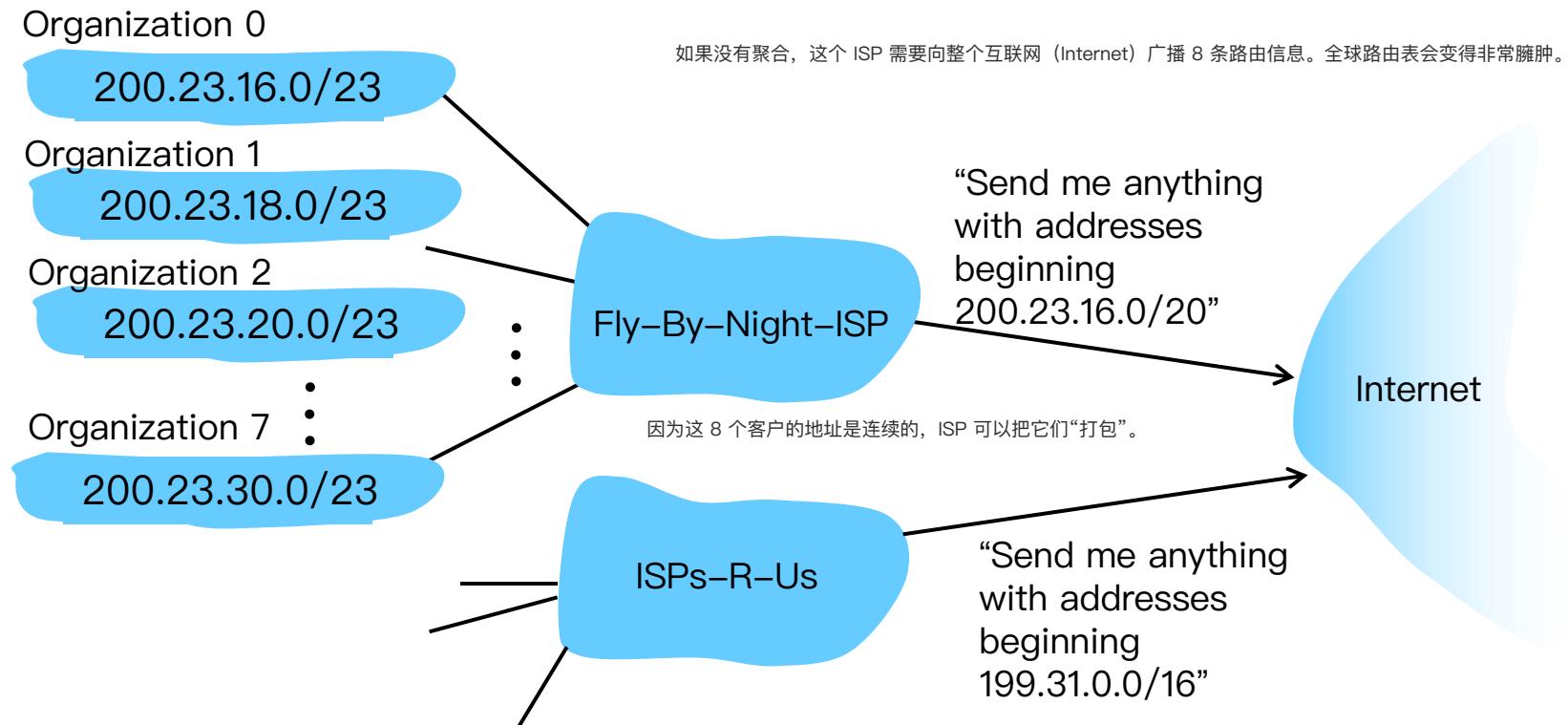
Subnet 3: since $2^5 = 32 > 28$, we can assign the following block

10100110 00000100 00010100 11100000

which can be represented as 166.4.20.224/27.

Hierarchical addressing: route aggregation

hierarchical addressing allows efficient advertisement of route information:



How does a host get an IP?

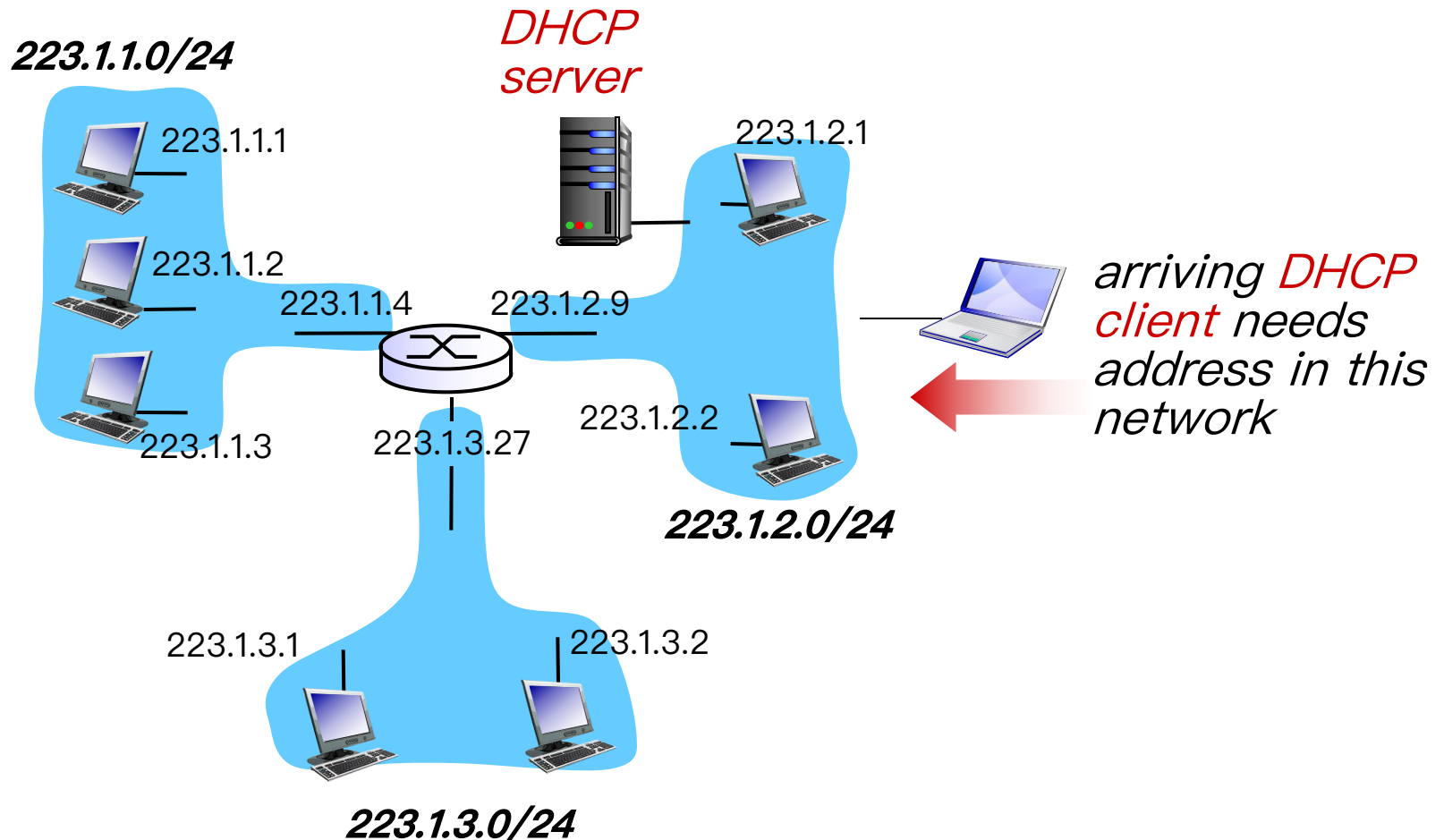
Q: How does a *host* get IP address?

- hard-coded by system admin in a file
 - Windows: control-panel->network->configuration->tcp/ip->properties
 - UNIX: /etc/rc.config
- **DHCP: Dynamic Host Configuration Protocol:** dynamically get address from as server
 - “plug-and-play”

主机启动时，向网络广播：“谁是 DHCP 服务器？我要个 IP！”

DHCP 服务器收到后，从地址池里租一个 IP 给主机，并告诉它掩码、网关和 DNS。

DHCP client-server scenario



DHCP: Dynamic Host Configuration Protocol

goal: allow host to **dynamically** obtain its IP address from network server when it joins network

- can renew its **lease** on address in use
- allows reuse of addresses (only hold address while connected/ “on”)
- support for mobile users who want to join network (more shortly)

DHCP overview:

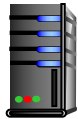
- host broadcasts “DHCP discover” msg
- DHCP server responds with “DHCP offer” msg
- host requests IP address: “DHCP request” msg
- DHCP server sends address: “DHCP ack” msg

DHCP client-server scenario

DHCP server: 223.1.2.5

DHCP discover

arriving client



Broadcast: is there a
DHCP server out there?

DHCP offer

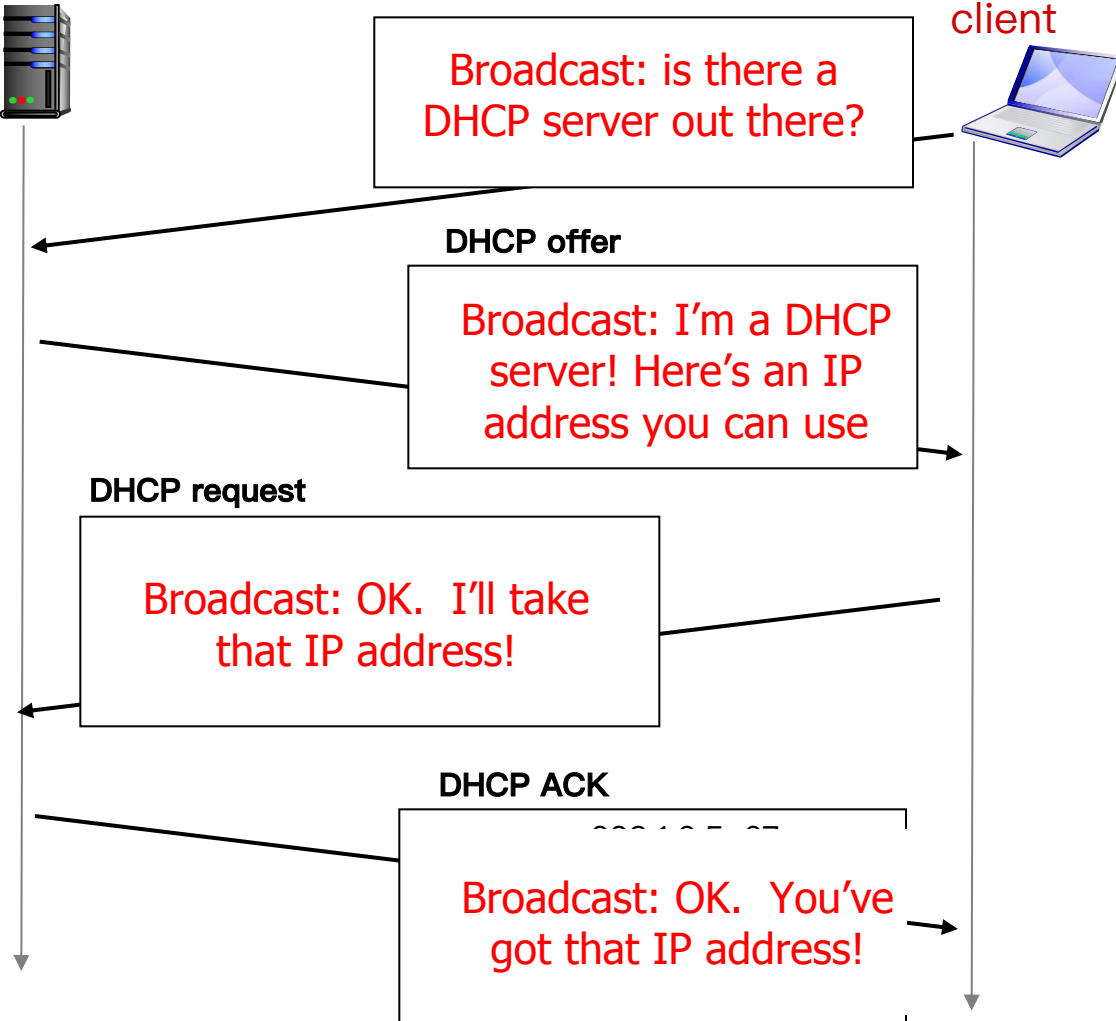
Broadcast: I'm a DHCP
server! Here's an IP
address you can use

DHCP request

Broadcast: OK. I'll take
that IP address!

DHCP ACK

Broadcast: OK. You've
got that IP address!



DHCP client-server scenario

DHCP server: 223.1.2.5

DHCP discover

src : 0.0.0.0, 68
dest.: 255.255.255.255, 67
yiaddr: 0.0.0.0
transaction ID: 654

arriving
client



DHCP offer

src: 223.1.2.5, 67
dest: 255.255.255.255, 68
yiaddr: 223.1.2.4
transaction ID: 654
lifetime: 3600 secs

DHCP request

src: 0.0.0.0, 68
dest.: 255.255.255.255, 67
yiaddr: 223.1.2.4
transaction ID: 654
lifetime: 3600 secs

DHCP ACK

src: 223.1.2.5, 67
dest: 255.255.255.255, 68
yiaddr: 223.1.2.4
transaction ID: 654
lifetime: 3600 secs

客户端收到这个 ACK 包后, 才正式把 IP 配置到自己的网卡上, 开始上网

DHCP: more than IP addresses

DHCP can return more than just allocated IP address on subnet:

- address of first-hop router for client
- name and IP address of DNS sever
- network mask (indicating network versus host portion of address)

默认网关。也就是“出门的钥匙”。没有它你出不去局域网。